

Windy

The Interface Is the Model

Evidence-Carrying Continuations for
Small-Model Static Binary Analysis

A terminal-hosted, evidence-first substrate that moves tool selection, state, verification, and provenance out of the model and into a deterministic local protocol.

RELEASE	DOMAIN	INTERFACE	STATUS
v0.3.0	Windows PE / MDMP	Six MCP tools	Final technical paper

DECISION BRIEF

Executive summary

RESEARCH QUESTION

Can a deterministic analytical substrate improve an untrained small language model's static reverse-engineering performance by removing protocol mechanics and returning only compact, verifiable evidence?

PAIRED BENCHMARK OUTCOMES / DEVELOPMENT SET

TASK SUCCESS	VALID CALLS	MEDIAN CALLS	TOOL OUTPUT
12/12 from 3/12 +75 points	100% from 82.81% +17.19 points	2 from 5.5 -63.6%	1,651 B from 5,314 B -68.9%

Bottom line

Windy v0.3 asks the analytical substrate to carry the protocol burden that a small model handles poorly. Its Evidence Query VM compiles intent into typed, demand-driven work and returns server-bound continuation tickets with compact, provenance-linked evidence deltas.

In the 12-case local benchmark, a fresh low-effort gpt-5.6-luna agent improved from 3/12 tasks with frozen v0.2 to 12/12 with v0.3. Valid calls reached 100%, median calls fell to 2, median tool output fell to 1,651 bytes, and supported-answer time fell from 46.32 to 9.44 seconds.

Abstract

Tool-using language models are usually improved by making the model larger, training it on tool traces, or supplying more context. This paper studies the opposite intervention: make the analytical substrate carry more of the protocol burden. Windy v0.3 is a terminal-hosted Model Context Protocol substrate for static Windows binary analysis.

The six-tool surface has a 3,170-byte schema. On a 53.9 MB self binary, mapped, catalog, and sketch stages reached p50 latencies of 79, 206, and 670 ms with 62.6 MiB peak RSS. A compact deep index used 85.9 MB for 10.74 million records. The architecture tests whether weak-model reverse engineering improves when tool use is compiled into evidence-carrying continuations.

Keywords: agentic binary analysis; small language models; MCP; demand-driven analysis; tool use; evidence provenance; continuation protocols

1. Introduction

Static reverse engineering is a hostile tool-use environment for a small language model. The model must select among many operations, bind virtual addresses and target identifiers, wait for analysis readiness, interpret partial results, preserve provenance, and avoid unsupported claims. Each obligation consumes context and creates a new failure boundary.

A conventional interface exposes more expert operations and asks the model to plan more carefully. Windy v0.3 instead asks which obligations can be removed from the model entirely. An investigation begins with a target or path, typed intent, natural-language question, and budget. The server compiles the request into deterministic operators, performs the cheapest useful analysis, and returns an evidence delta with at most three opaque actions.

A common continuation requires only an `action_id`; target, revision, arguments, readiness, cost, and expiry are already bound. Large results are immutable paged artifacts rather than prompt payloads. The model still supplies intent and interprets evidence, while mechanics are expressed in code, type checks, budgets, and revision guards.

Contributions

- An Evidence Query VM that compiles investigative intent into typed, deterministic static-analysis operators without embedding a planner or model.
- Evidence-carrying continuations that bind analytical state and return compact deltas with support, uncertainty, omissions, and stable evidence identifiers.
- A demand-driven analysis lattice and compact function-sketch domain that postpone expensive analysis until requested.
- A deterministic microbenchmark showing a 75-point development-set gain with fewer calls, fewer schema bytes, and less tool output.

2. Related Work and Design Position

Weak-model tool use

Shen et al. split tool use into planning, calling, and summarization because one small model struggles to learn all roles [1]. Windy moves that split across a systems boundary: intent and synthesis remain external; selection, binding, readiness, budgets, and evidence bookkeeping become deterministic server work.

Tool retrieval and context

ToolLLM and Re-Invoke motivate compact capability selection [2, 3], but retrieval still leaves arguments and state transitions to the caller. Windy converts selected work into prevalidated actions. Lost in the Middle motivates short evidence deltas, stable paging, 2 KiB observations, and server-side deduplication [4].

Demand analysis and binary analysis

Demand analysis computes facts required by a query instead of solving a whole program eagerly [5]. Windy applies that discipline across six analysis stages. BINREX demonstrates semantic retrieval, task adaptation, and verifiable reasoning for agentic binary analysis [6]. Windy asks whether a compact, model-agnostic substrate can make a weak caller more reliable and efficient.

What is potentially novel

The ingredients are established: demand analysis, tool retrieval, capabilities, state machines, abstract interpretation, provenance, and static verification. Windy combines them in a reverse-engineering MCP server that issues executable, state-bound continuations carrying evidence identifiers and verification state. The contribution is the architecture and its measured behavior.

3. Evidence Query VM

Windy's Evidence Query VM is the boundary between a natural-language investigation and typed static-analysis work. It binds identifiers, chooses the cheapest useful operator, preserves revision and budget state, verifies requested high-confidence claims, and emits compact evidence rather than whole-image dumps.

Demand-driven analysis lattice

Only the stage required by an investigation is promoted

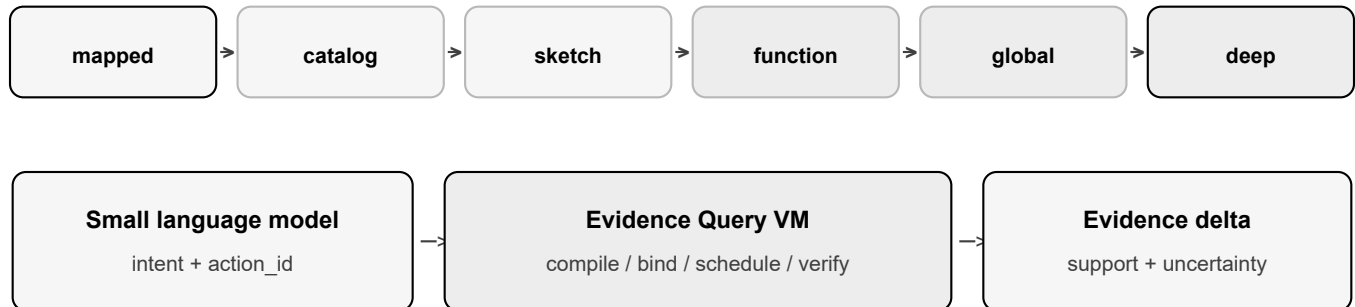


Figure 1. The model states intent; the server compiles mechanics and returns bounded evidence. Analysis is promoted only as demanded.

Public protocol

Tool	Contract
windy_status	Inspect target, investigation, job, cache, or runtime state.
investigation_start	Open or reuse a target; compile intent, question, and budget.
investigation_step	Execute a server-issued action ticket, commonly with action_id only.
evidence_read	Page immutable evidence and large artifacts.
change_commit	Commit a verified proposal with revision and idempotency guards.
target_close	Flush annotations, cancel target work, and close the session.

Table 1. The complete always-advertised Windy v0.3 tool surface.

The interface intentionally breaks with v0.2. Specialized decompilation, SSA, dump, workspace, history, vtable, cross-project, and index operations remain internal operators or resource-backed artifacts. Action tickets name the exact operation but remain opaque to the caller and cryptographically bound to investigation state. Expiry, tampering, and revision conflicts return machine-readable repair choices instead of silently changing meaning.

Typed intents and deterministic compilation

The compiler accepts nine intents: locate, explain, trace, verify, read_data, compare, edit, capability, and dump. Natural-language terms remain lexical evidence. A deterministic ontology rewrites phrases into constraints over sketches, metadata, relationships, and proof obligations. Constraint intersection ranks candidates; structural analysis verifies requested high-confidence claims.

PROTOCOL SHAPE

```
investigation_start(path, intent, question,
budget)
-> investigation_id, state, evidence_delta,
[action_id ...]

investigation_step(investigation_id, action_id)
-> new_evidence_only, completeness, uncertainty,
[action_id ...]
```

Machine-readable repair

Weak callers fail predictably. Invalid calls return a small set of legal repairs instead of a prose error dump. When a requested path is absent, Windy may recover one uniquely missing ASCII filename character; ambiguous repairs fail closed. The mechanism is generic, evaluation-derived, and covered by negative tests.

Evidence-carrying continuations

Evidence-carrying describes a protocol property, not a formal proof calculus. Each delta refers to stable evidence records with provenance, target revision, completeness, uncertainty, and omitted counts. The server retains prior evidence and returns only new material.

Continuations are capability-like: possession authorizes one prevalidated transition, while revision and expiry constrain replay. Mutating continuations additionally require idempotency keys and optimistic revision checks. A valid action therefore cannot be redirected to a different target or reinterpret stale arguments.

CORE PROTOCOL IDEA

Move selection, argument binding, readiness, state, and evidence bookkeeping into the substrate. Leave investigative intent and evidence synthesis with the external model.

4. Demand-Driven Static Analysis

Stage	Retained work	Promotion trigger
mapped	Memory map, image validation, essential identity	Target open
catalog	Headers, sections, imports, exports, unwind/PDB seeds	Triage or metadata query
sketch	Streaming decode into compact per-function semantic facts	Intent ranking
function	CFG, stack, calls, SSA, types, native decompilation window	Candidate verification or inspect
global	Incoming xrefs, string refs, indirect edges, cross-function indexes	Trace or global query
deep	Partitioned instruction, token, numeric, and motif indexes	Explicit deep request

Table 2. Monotone stage promotions make readiness explicit and prevent eager whole-image work.

5. Failure-Driven Evaluation

Function sketches and compact indexing

The sketch stage retains ranking facts - control-flow shape, calls, strings, imports, constants, memory behavior, dependencies, side effects, and semantic motifs - while reconstructing full decoder objects only for active function windows.

Deep indexing stores sorted eight-byte instruction records instead of full instruction objects and per-instruction maps. Checksummed, ABI-keyed partitions are atomically written, memory-mappable, corruption recoverable, and bounded by a 5 GiB disk LRU. Mutable annotations remain separate.

Scheduling and memory

A four-worker priority pool orders foreground work, cache loads, warming, and optional indexing. Single-flight deduplication and bounded waits prevent duplicate work and zero-work polling. Analysis partitions and decoded windows share a 1 GiB process budget.

Microbenchmark

The external SQLite evaluator generates three neutral PE programs in P0 and P2 variants, then stages stripped binaries without source, symbols, descriptive filenames, or adjacent gold. Twelve deterministic cases cover location, explanation, relationships, provenance, data, types, verification, abstention, and edit persistence.

Each case used a fresh low-reasoning gpt-5.6-luna agent with only the task, sanitized path, MCP endpoint, schemas, and budgets. Limits were six calls, 8 KiB cumulative tool output, and a 250-token answer. Source or gold inspection invalidated a run.

Iteration rule

Failures were classified by stage. One mechanism changed at a time when practical and remained only when it improved a canary or reduced context or latency by 15% without degrading quality or honest abstention.

Observed failure	Mechanism introduced	Protocol effect
Hidden operation selection	Typed intent compiler	Model chooses task class, not low-level tool
Malformed capability arguments	Server-bound action ticket	Arguments are prevalidated and cannot drift
Intent lost during canonicalization	Retained descriptive lexemes	Ranking keeps task-specific constraints
Ambiguous data-structure match	Constraint intersection over sketches	Pointer walk and side effects jointly rank
Zero-work continuation poll	Bounded foreground wait	A charged call performs useful progress

Table 3. Failure episodes and general mechanisms retained in v0.3.

6. Results

Paired development-set outcomes

FROZEN V0.2 -> WINDY V0.3

TASK SUCCESS	VALID CALLS	MEDIAN CALLS	TOOL OUTPUT	ANSWER TIME
25.0%	82.81%	5.5	5,314 B	46.32 s
100%	100%	2	1,651 B	9.44 s
+75 points	+17.19 points	-63.6%	-68.9%	4.9x faster

Figure 2. Exact paired before/after values replace mixed-scale bars.

Metric	Frozen v0.2	Windy v0.3	Change
Cases passed	3/12 (25.0%)	12/12 (100%)	+75.0 points
False support	0	0	No regression
Valid public calls	82.81%	100%	+17.19 points
Median MCP calls	5.5	2	-63.6%
Median tool output	5,314 B	1,651 B	-68.9%
p95 tool output	9,101 B	4,443 B	-51.2%
Median supported-answer time	46.32 s	9.44 s	4.9x faster

Table 4. Paired local results. Timing uses the corrected v0.2 supported-case replay.

The agent stopped attempting legacy tools, stopped assembling raw capability arguments, and received evidence sized for the question. The 12/12 run had zero false support and 100% valid public calls. This is development-set evidence that the mechanisms eliminate observed failures.

Schema and context surface

Surface	Frozen v0.2	v0.3	Reduction
tools/list JSON	7,343 B	3,170 B	56.8%
Always-advertised tools	12	6	50.0%
Default inline budget	4 KiB	2 KiB	50.0%
Hard inline budget	64 KiB	8 KiB	87.5%

Table 5. The model-visible interface was reduced while internal capability parity was retained.

Runtime and software verification

Target	Cold mapped	Cold catalog	Cold sketch	Warm mapped	Warm catalog	Warm sketch	Peak RSS
sample.exe	21 ms	28 ms	50 ms	3 ms	9 ms	30 ms	12.5 MiB
complex.exe	21 ms	27 ms	51 ms	3 ms	9 ms	28 ms	12.5 MiB
Windy self (53.9 MB)	79 ms	206 ms	670 ms	3 ms	8 ms	53 ms	62.6 MiB

Table 6. Five-run p50 release measurements. Peak RSS is the maximum working set observed by the harness.

The self binary met every runtime and memory gate. Its deep index retained 10,740,446 records in 85,923,568 bytes; five fresh-cache builds had a 279 ms median. Versus the earlier eager BEL observation of 202 seconds and 2.28 GB, it was about 724 times faster and 96.2% smaller by estimated index memory; the indexes are not feature-identical. A minimal MDMP parsed in 12-19 ms; rich traversal was not measured.

Gate	Observed result
Build and lint	cargo build passed; clippy passed with zero warnings
cargo test	431 passed, 0 failed, 1 authoring helper ignored; 30.76 s
python -m unittest discover eval/microbench	4/4 passed in 0.22 s
cargo build --release	Pass; 53,860,864-byte executable
Dependency assertion	No eframe, egui, rfd, wgpu, or winit dependency

Table 7. Mandatory local verification completed before manuscript generation.

7. Why the Architecture Helps

Executable inductive bias

In v0.2, the caller had to infer a hidden state machine: bind a capability, discover readiness, paginate, preserve citations, and judge support. In v0.3, these requirements are executable server state. The caller states intent, chooses a valid next action, and synthesizes bounded evidence.

Programming-languages ideas

Tickets resemble capabilities; revision and expiry resemble typestate; evidence deltas resemble incremental views; sketches resemble abstract interpretation; and promotion resembles demand-driven dataflow. These analogies provide a design vocabulary and test obligations without implying formal verification.

Honesty through explicit incompleteness

Binary analysis is incomplete by construction. Indirect calls, obfuscation, missing symbols, malformed images, and unresolved data can prevent proof. Windy makes completeness explicit. Partial and pending states carry reasons, omissions, stable cursors, and uncertainty. Verification classifies claims as supported, contradicted, or unknown; honest abstention succeeds when support is unavailable.

CORE HYPOTHESIS

For weak tool callers, moving selection, binding, scheduling, state, and provenance into a deterministic substrate can yield a larger practical gain than exposing more tools or context. Fresh tasks must preserve quality, honesty, and efficiency.

8. Evaluation Boundaries

Development data

The 12 cases were used during failure-driven iteration. They measure the mechanisms fixed in v0.3; transfer to unseen work requires a fresh sealed run.

Scope

The study covers six small Windows PE fixtures and one low-reasoning model configuration. The MDMP result is a parser smoke, not a rich-dump benchmark.

Measurement

Response bytes stand in for unavailable token telemetry. The 4.9x timing uses the corrected supported-case replay rather than a matched all-case sample.

Attribution and semantics

v0.3 combines several mechanisms without a factorial ablation. Tickets reduce argument drift; evidence IDs prove provenance, not semantic truth. Decompilation, recovered types, and function boundaries remain best-effort.

9. Next Validation

Freeze the current executable and schema, then run at least 60 unseen sealed cases across compilers, optimization levels, and balanced abstention tasks.

- Compare frozen v0.2, full v0.3, and targeted ablations under identical model settings and budgets.
- Record deterministic success, false support, valid calls, tokens, time to first supported answer, verified facts, cache behavior, latency, memory, and artifact bytes.
- Commit hashes and answers before revealing gold; report paired differences and bootstrap confidence intervals.

10. Conclusion

Windy v0.3 tests a simple proposition: improving an agent does not always require improving its language model. The Evidence Query VM compiles intent into typed static-analysis work, issues state-bound continuations, and returns only new, provenance-linked evidence. In the local benchmark, the redesign converted repeated protocol failures into 12/12 successful tasks while reducing calls, context, latency, and analysis cost.

The core contribution is architectural: treat the interface as executable inductive bias, continuation state as a server responsibility, and evidence completeness as part of the contract. The measured benchmark is development data; independent replication is the next test. The design already provides a practical route to capable, private, low-cost agents without training or fine-tuning.

Credits

This work was conducted under Husena LLC. Demetrius Greses Jr. directed the project and is the accountable human author. OpenAI Daybreak Blue assisted architecture synthesis, implementation iteration, experimental analysis, and manuscript preparation. OpenAI gpt-5.6-luna was the blinded low-reasoning benchmark subject. AI-generated assistance was reviewed and selected by the human author; AI systems are not listed as human authors. Demetrius Greses Jr. retains responsibility for the claims, interpretation, disclosure, and release of this work.

References

- [1] W. Shen et al. *Small LLMs Are Weak Tool Learners: A Multi-LLM Agent*. EMNLP 2024, pp. 16658-16680. doi:10.18653/v1/2024.emnlp-main.929.
- [2] Y. Qin et al. *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs*. arXiv:2307.16789, 2023. arxiv.org/abs/2307.16789.
- [3] Y. Chen et al. *Re-Invoke: Tool Invocation Rewriting for Zero-Shot Tool Retrieval*. Findings of EMNLP 2024, pp. 4705-4726. doi:10.18653/v1/2024.findings-emnlp.270.
- [4] N. F. Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. TACL 12, 2024, pp. 157-173. doi:10.1162/tacl_a_00638.
- [5] E. Duesterwald, R. Gupta, and M. L. Soffa. *Demand-Driven Computation of Interprocedural Data Flow*. POPL 1995. doi:10.1145/199448.199461.
- [6] Y. Liu et al. *Towards Generality: Task-Adaptive Binary Analysis via Semantic Retrieval and Verifiable Reasoning*. 35th USENIX Security Symposium, 2026. USENIX open-access page.
- [7] Y. Hao et al. *ToolkenGPT: Augmenting Frozen Language Models with Massive Tools via Tool Embeddings*. NeurIPS 2023. arxiv.org/abs/2305.11554.
- [8] A. Kambhampati et al. *LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks*. arXiv:2402.01817, 2024. arxiv.org/abs/2402.01817.
- [9] Model Context Protocol. *Tools specification*. Model Context Protocol specification. Accessed 2026-08-30.
- [10] T. Bao et al. *BYTEWEIGHT: Learning to Recognize Functions in Binary Code*. 23rd USENIX Security Symposium, 2014. USENIX open-access page.